

Global Sales & Retail Performance Analytics System

Project Overview

This project delivers an end-to-end retail sales analytics system that integrates data engineering, workflow orchestration, and business intelligence to analyze sales performance, customer behavior, product profitability, and regional trends.

The system follows a **Medallion Architecture (Bronze → Silver → Gold)** to transform raw retail data into analytics-ready datasets and interactive dashboards for business stakeholders.

Introduction

- Retail sales data consists of the information generated when products are sold to customers by a retail organization. This data includes details related to sales transactions, products, customers, stores, and regions. Retail businesses generate large volumes of such data on a daily basis through both online and offline sales channels.
- Retail sales analytics is the process of organizing and analyzing this data to understand business performance. By analyzing sales data, businesses can identify sales trends, understand customer purchasing behavior, evaluate product performance, and compare performance across different regions and stores.
- Retail sales analytics helps organizations transform raw sales data into meaningful insights, which support better planning, performance monitoring, and informed business decision-making.

Objectives

- Build an end-to-end analytics pipeline
- Automate data ingestion and transformation
- Improve data quality and consistency
- Generate meaningful business KPIs
- Create interactive Power BI dashboards
- Demonstrate real-world ETL and orchestration

Business Context

Retail organizations generate large volumes of transactional data across multiple regions, products, and customer segments. Without a centralized analytics pipeline, this data becomes difficult to process, validate, and analyze consistently.

This project demonstrates how modern data engineering tools can be used to automate data processing, improve data quality, and generate actionable business insights.

Datasets Used

sales_transactions.csv

- **10150 rows**

- Order-level transactional data
- Contains order ID, date, product, quantity, pricing, discounts, and revenue-related fields

products.csv

- **306 rows**
- Product master data
- Includes product name, category, sub-category, cost price, and selling price

customers.csv

- **2040 rows**
- Customer master data
- Includes customer ID, segment, and demographic attributes

stores.csv

- **50 rows**
- Store and regional reference data
- Maps stores to regions and locations

returns.csv

- **800 rows**
- Returned order records
- Includes return reason and refund amount

sales_targets.csv

- **48 rows**
- Sales targets defined by region and time period
- Used for target vs actual performance analysis

Data Processing & ETL Layers

- Medallion Architecture is a data design pattern used in modern data engineering to organize data into multiple layers based on data quality and processing stage.
- Bronze Layer stores raw data ingested from source systems without modification
- Silver Layer contains cleaned, validated, and standardized data
- Gold Layer holds aggregated, business-ready datasets used for analytics and reporting

- This architecture improves data quality, scalability, maintainability, and enables clear separation between raw data processing and business analytics.

What is Databricks?

Databricks is a cloud-based data analytics platform built on Apache Spark that enables large-scale data processing, transformation, and analytics using distributed computing.

Why Databricks is Used

- Provides scalable processing for large retail datasets
- Supports PySpark and Spark SQL for efficient data transformations
- Enables implementation of the Medallion Architecture (Bronze, Silver, Gold)
- Integrates Delta Lake for reliable and consistent data storage
- Supports interactive notebooks for development and debugging

Databricks helps efficiently transform raw retail data into analytics-ready datasets for reporting and decision-making.

Bronze Layer – Raw Data Ingestion

- Reads raw CSV files from Databricks Volumes
- Preserves original schema and values
- Stores data as Delta tables
- No transformations applied

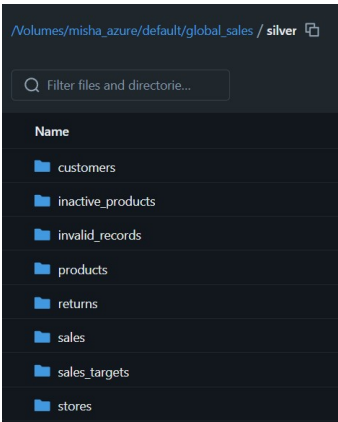
```
root
|-- order_id: string (nullable = true)
|-- order_date: date (nullable = true)
|-- customer_id: string (nullable = true)
|-- product_id: string (nullable = true)
|-- store_id: string (nullable = true)
|-- region: string (nullable = true)
|-- sales_channel: string (nullable = true)
|-- quantity: integer (nullable = true)
|-- unit_price: integer (nullable = true)
|-- discount: integer (nullable = true)
|-- payment_method: string (nullable = true)
|-- order_status: string (nullable = true)
|-- ingestion_ts: timestamp (nullable = true)
|-- source_file: string (nullable = true)
```

Silver Layer – Data Cleaning & Validation

Key preprocessing steps include:

- Duplicate record removal
- Handling missing and invalid values
- Foreign key validation across datasets

- Standardization of date and category formats
- Creation of derived metrics
- Separation of invalid records into dedicated tables



Invalid Records Handling and Data Quality Management

- Data quality checks were applied during Silver-layer processing to identify invalid records, including missing keys, incorrect references, invalid dates, and incorrect pricing values. Records failing validation rules were separated into dedicated invalid tables, while clean records continued through the pipeline.
- This approach ensured that analytics were generated only from reliable data while retaining invalid records for audit and future correction, improving overall data quality and pipeline reliability.

Sales_transactions

order_id	order_date	customer_id	product_id	store_id	region	sales_channel	quantity	unit_price	discount
0100001	2025-07-31T00:00:00.000Z	C5745	P1046	S247	NORTH	STORE	3	-11020	498
0100006	2024-10-28T00:00:00.000Z	C6361	P1203	S239	WEST	STORE	-2	17152	0
0100008	2024-03-31T00:00:00.000Z	C5565	null	S215	EAST	STORE	2	13862	0
0100014	2025-06-19T00:00:00.000Z	C6448	null	S231	NORTH	STORE	5	11470	0
0100016	2024-04-24T00:00:00.000Z	C6329	P1290	S240	NORTH	ONLINE	4	-5987	0
0100020	2024-02-03T00:00:00.000Z	C5382	null	S200	WEST	STORE	5	10169	0

Customers

customer_id	customer_name	gender	age	city	state	country	customer_segment
C5254	null	Female	37	Jamnagar	Uttar Pradesh	India	CORPORATE
C5099	null	Female	39	Ranchi	Telangana	India	CORPORATE
C5029	null	Female	39	Bidar	Tripura	India	RETAIL
C5149	Bhamini Khurana	Male	130	Amravati	Telangana	India	CORPORATE
C6791	Dishani Dewan	Male	130	Nanded	Punjab	India	RETAIL

Gold Layer – Analytics & Aggregation

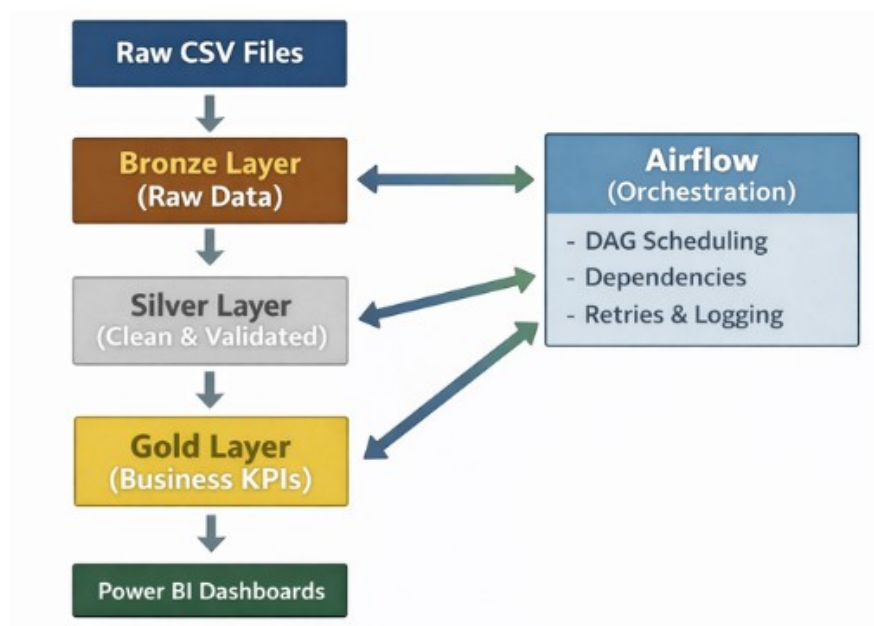
The Gold layer contains business-ready analytics tables:

- Gold tables were designed to represent distinct business perspectives and analytics use cases. Each table serves a specific analytical purpose and is optimized for reporting.
- The Sales Performance table provides overall sales metrics across time periods, including revenue, profit, and sales trends.
- The Product Performance and Profitability table analyzes product-level performance, costs, revenue, and margins.
- The Regional and Store-Level Performance table evaluates geographic performance across regions and stores.
- The Customer Purchase Analytics table captures customer behavior, purchase frequency, and contribution to revenue.
- The Target versus Actual Performance table compares actual sales metrics against predefined sales targets to measure performance achievement.

Tables (5)

- 📊 gold_customer_analytics
- 📊 gold_product_performance
- 📊 gold_region_performance
- 📊 gold_sales_performance
- 📊 gold_target_vs_actual

Architecture



Technology Stack

- **Data Processing:** Python, Pandas
- **Big Data & Analytics:** Azure Databricks, PySpark
- **Storage:** CSV files, Delta Tables
- **Workflow Orchestration:** Apache Airflow
- **Visualization:** Power BI Desktop

Project Structure & Execution Workflow

Project Structure

- **datasets/** Raw input retail datasets (CSV files)
- **databricks/** Databricks notebooks organized by Medallion Architecture:
 - **01_bronze_ingestion/**
 - **02_silver_transformation/**
 - **03_gold_analytics/**
- **airflow/** Apache Airflow setup and DAGs:
 - Dockerfile
 - docker-compose.yaml
 - dags/retail_sales_databricks_dag.py
- **powerbi/** Power BI dashboard files (.pbix)
- **README.md** Project documentation
- **presentation/** Final project presentation files

Execution Process

1. Upload Input Data to Databricks Volumes

1.1 Upload Raw CSV Files

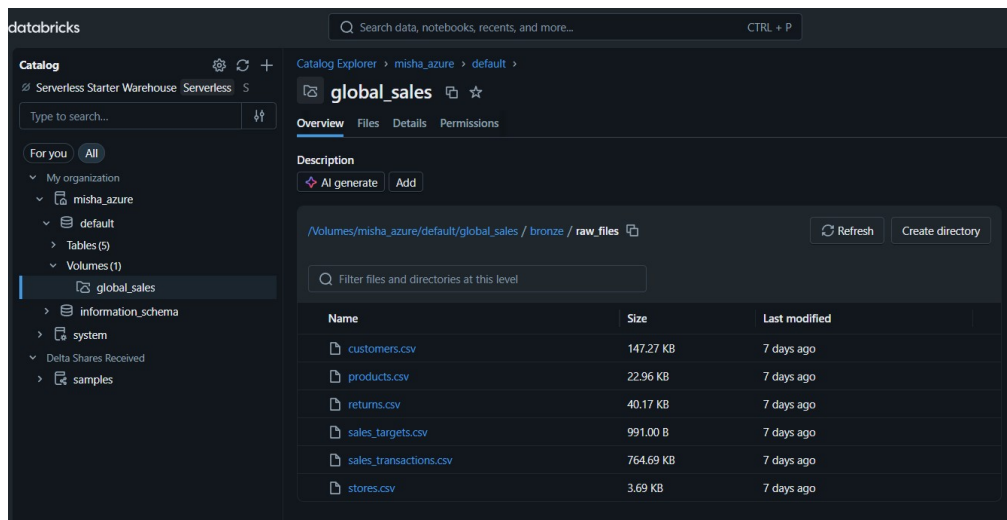
Before running any Databricks notebooks, the raw retail CSV files must be uploaded to Databricks storage.

Steps:

1. **Navigate to Databricks → Catalog → Volumes**
2. **Select an existing catalog and schema, or create one**
3. **Create the following folder structure:**

/Volumes/misha_azure/default/global_sales/data/

4. Upload all raw CSV files from the local datasets/ folder into this volume path



1.2 Update File Paths in Notebooks

All Databricks notebooks are configured to read input data from the Databricks volume path.

Example:

```
BASE_PATH = "/Volumes/misha_azure/default/global_sales/data/sales_transactions.csv"
```

2. Databricks Job Setup

2.1 Import Analytics Notebooks

Import the Databricks notebooks into your workspace.

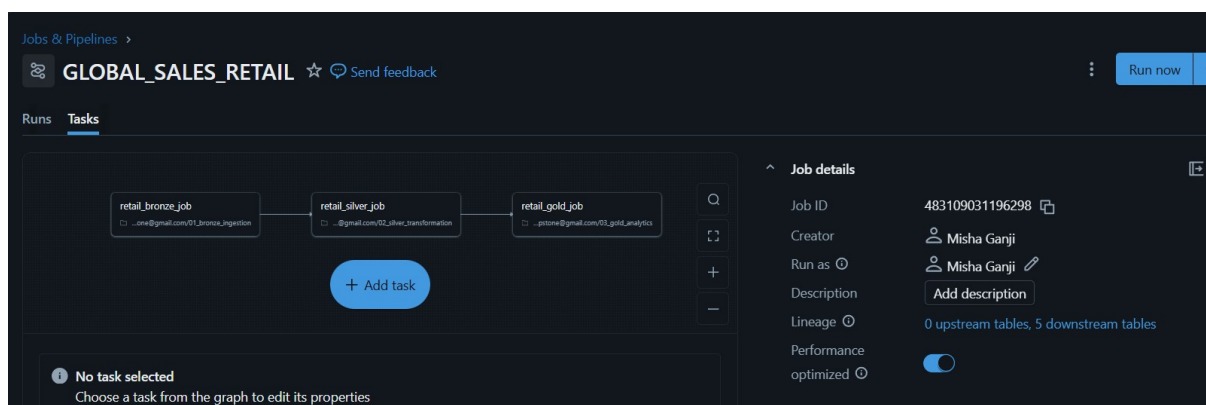
2.2 Create Databricks Job

1. Go to Databricks > Jobs & Pipelines > Create Job

2. Add three tasks in order (Bronze > Silver > Gold) using notebook paths from your workspace.

3. Attach a cluster

4. Save the job and note the Job ID



3. Airflow Setup Using Docker

3.1 Navigate to Airflow Project Directory

```
cd airflow
```

3.2 Configure Airflow Docker Environment

The Databricks provider is installed using Docker environment variables. Ensure the following configuration is present in docker-compose.yml:

```
_PIP_ADDITIONAL_REQUIREMENTS: apache-airflow-providers-databricks
```

This enables Airflow to communicate with Databricks Jobs.

3.3 Start Airflow Services

```
docker compose up -d
```

This command starts all required Airflow components

3.4 Access Airflow UI

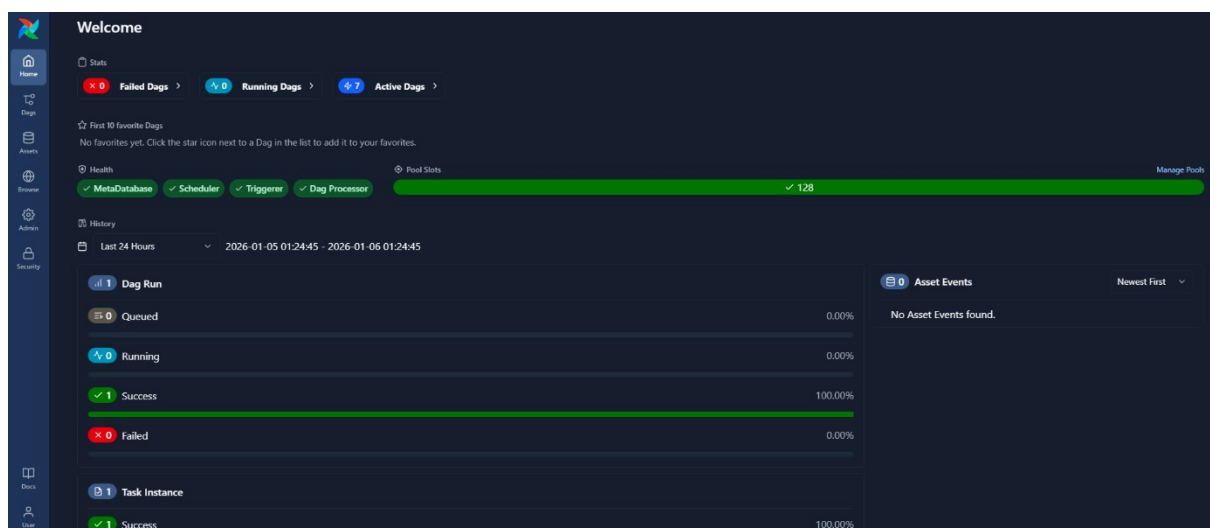
Open the Airflow web interface in a browser:

<http://localhost:8080>

Login using the default credentials configured in Docker:

Username: airflow

Password: airflow



3.5 Configure Databricks Connection in Airflow

To allow Airflow to trigger Databricks jobs:

- Navigate to Admin → Connections
- Click Add Connection

- Configure the connection as follows:

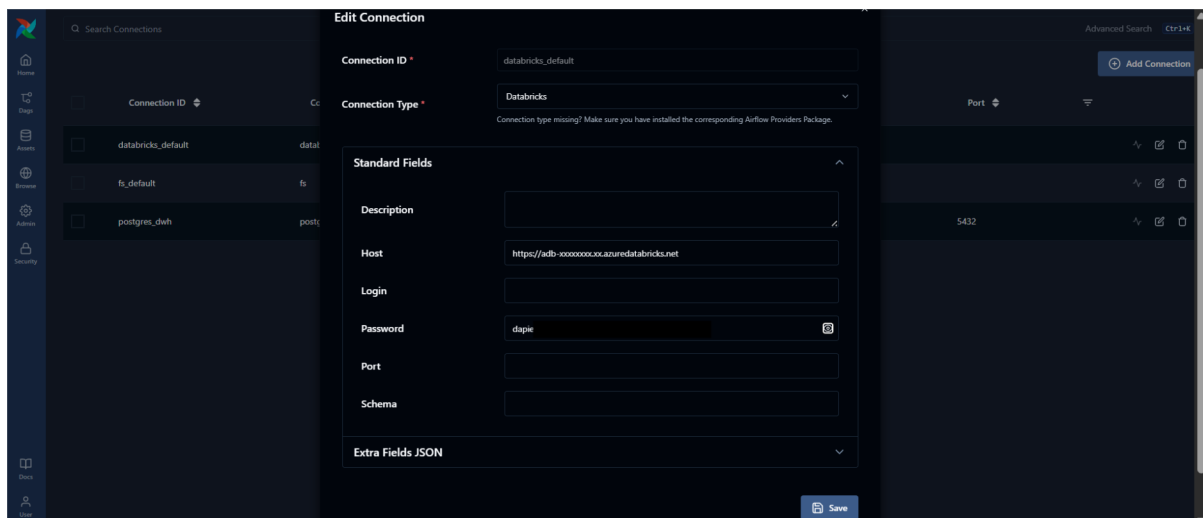
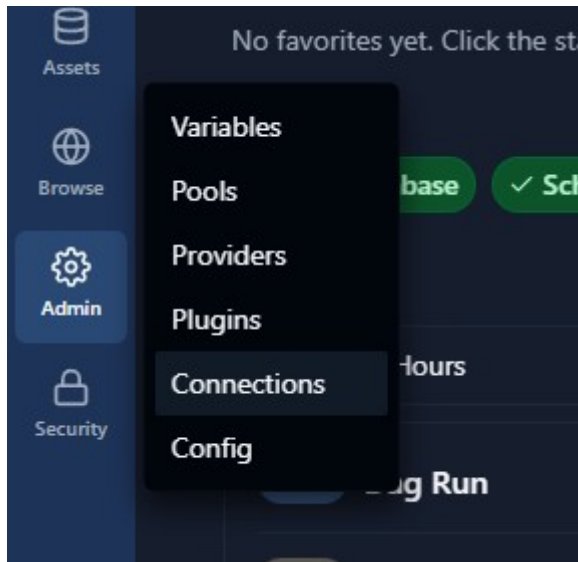
Conn Id: databricks_default

Conn Type: Databricks

Host: https://adb-<workspace-id>.<region>.azuredatabricks.net

Password: <Databricks Personal Access Token>

This connection is used by Airflow DAGs to securely invoke Databricks Jobs.



3.6 Create Airflow DAG for ETL Orchestration

An Airflow DAG file is created to define the ETL workflow and task dependencies.

The DAG file is placed inside the dags/ directory.

The DAG code defines:

- Pipeline structure
- Task dependencies

- Databricks job triggers
- Retry and failure handling

DAG File Location:

dags/retail_sales_databricks_dag.py

DAG Responsibilities:

- Orchestrates Bronze → Silver → Gold execution order
- Triggers separate Databricks jobs for each layer
- Ensures reliable and repeatable pipeline execution

3.7 Deploy and Trigger the DAG

Once the DAG file is added, Airflow automatically detects it. If required, containers can be restarted to refresh the environment.

`docker compose up -d`

The DAG named **retail_sales_end_to_end_pipeline** appears in the Airflow UI.

The pipeline should be triggered and run

Execution order enforced by the DAG:

Start → Bronze → Silver → Gold → End

3.8 Verify Pipeline Execution

Monitor execution using:

- Graph View
- Task Logs

Successful execution confirms:

- Correct task orchestration
- Successful Databricks job invocation
- End-to-end ETL automation

The screenshot shows the Databricks Jobs & Pipelines interface. On the left, a sidebar contains navigation icons for Home, Dags, Assets, Browse, Admin, Security, Docs, and User. The main area displays a DAG for 'trigger_databricks_retail_job' with a 'DatabricksRunNowOperator' task. A blue box highlights the task with a green 'success' status. On the right, a 'Task' panel shows details for the 'trigger_databricks_retail_job' task, including the operator 'DatabricksRunNowOperator', start and end dates, duration, and version. Below this, a 'Logs' panel shows a list of log messages, including 'DAG bundles loaded', 'Task instance is in running state', and 'Run submitted with run_id: 1044818073103971'.

The screenshot shows the Databricks Jobs & Pipelines interface for a job named 'GLOBAL_SALES_RETAIL'. The top bar includes 'Jobs & Pipelines > GLOBAL_SALES_RETAIL >' and a 'Send feedback' button. Below this, a 'Delete job run' and 'Repair run' button are visible. The main area displays a DAG with three tasks: 'retail_bronze_job', 'retail_silver_job', and 'retail_gold_job'. The 'retail_bronze_job' task is highlighted with a green box. On the right, a 'Job run details' panel shows information for the job run, including Job ID, Job run ID, Launched status, Started and Ended times, Duration, Execution time, Queue duration, Status (Succeeded), Lineage (0 upstream tables, 5 downstream tables), Performance (Enabled), and optimization. A 'View run events' button is also present.

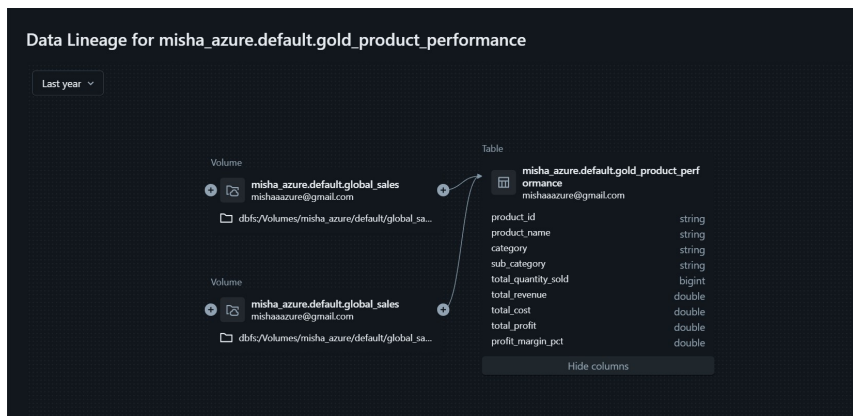
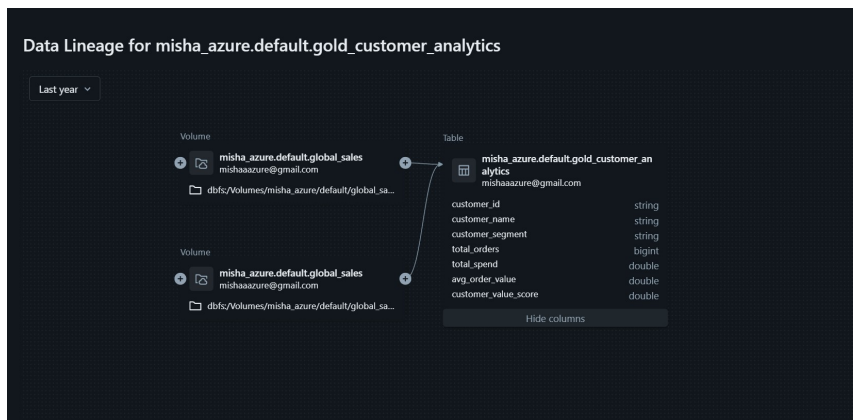
(or)

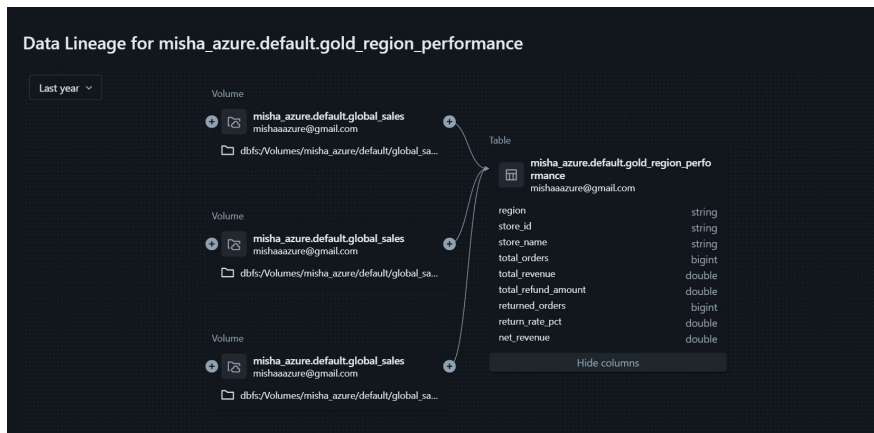
We can create multiple jobs for individual layers to get following DAG

The screenshot shows the Databricks Jobs & Pipelines interface for a job named 'etl_pipeline_sales_retail'. The top bar includes 'Dag', 'Dag Run', and 'Task' tabs. The main area displays a DAG with five tasks: 'start_pipeline', 'bronze_ingestion', 'silver_transformation', 'gold_aggregation', and 'end_pipeline'. The 'gold_aggregation' task is highlighted with a green box. On the right, a 'Task' panel shows details for the 'gold_aggregation' task, including the operator 'DatabricksRunNowOperator', start and end dates, duration, and version. Below this, a 'Logs' panel shows a list of log messages, including 'DAG bundles loaded', 'Task instance is in running state', and 'Run submitted with run_id: 110086488998576'.

Data Lineage

- Data lineage was used to track how data flows across the Medallion Architecture from raw source files to analytics-ready Gold tables. Lineage graphs were generated for all five Gold tables to verify correct dependencies between Bronze, Silver, and Gold layers.
- Each Gold table clearly shows its upstream sources, including cleaned Silver tables, raw Bronze tables, and the original CSV files stored in Databricks Volumes. This ensured transparency, traceability, and easier impact analysis across the pipeline.
- Lineage validation was performed for sales performance, product profitability, regional performance, customer analytics, and target versus actual performance tables. Screenshots of these lineage graphs are included as project evidence.





4. Gold Layer Consumption & Power BI Dashboard Creation

4.1 Create Gold Tables in Databricks

The final analytics tables are created in the **Gold layer** as Delta tables within the Databricks Catalog.

These tables contain business-ready KPIs and aggregated metrics optimized for reporting and visualization.

Once the Gold tables are successfully created, they are registered in the **Catalog** → **Tables** section.

4.2 Connect Microsoft Power BI to Azure Databricks Using SQL Warehouse

Power BI is connected to Databricks using the **Databricks SQL Warehouse (Serverless)** connection.

Steps:

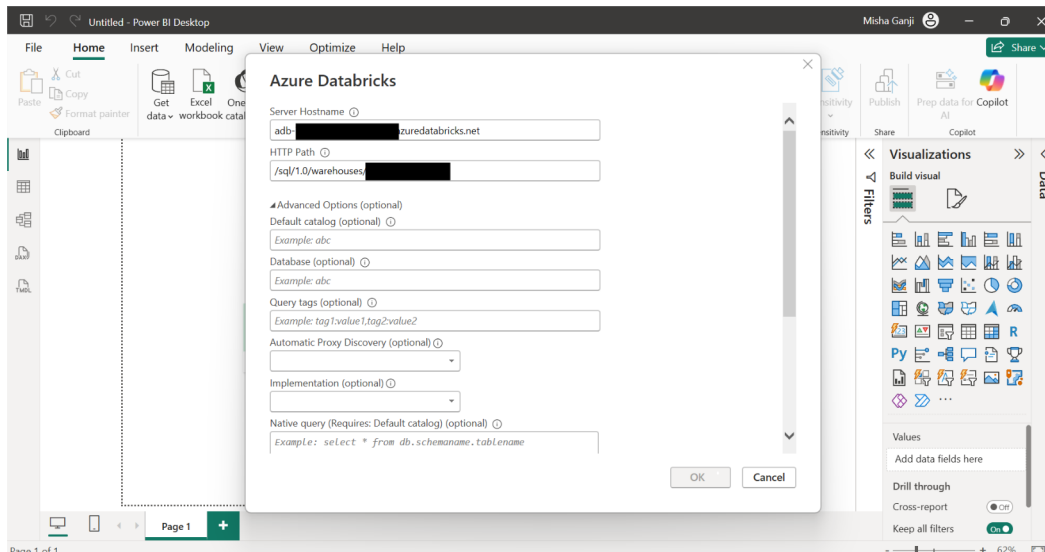
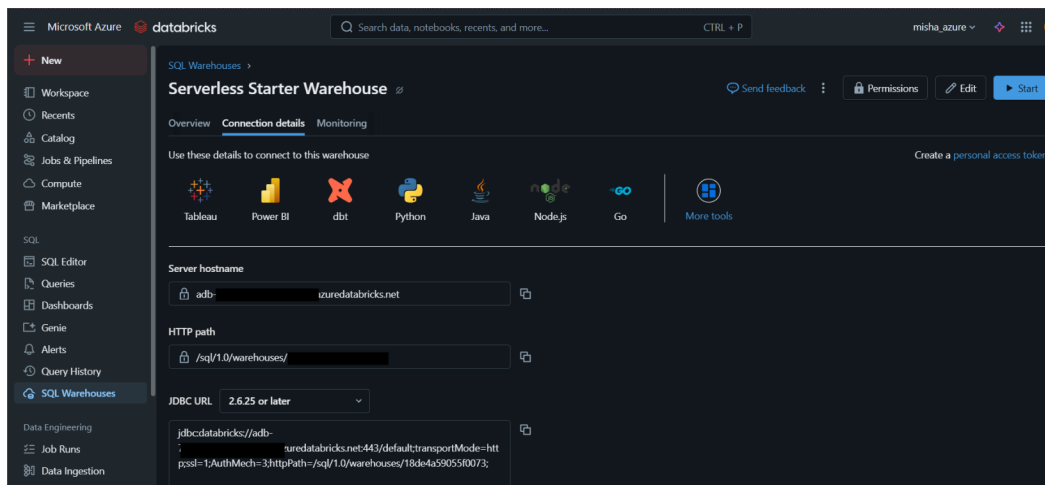
1. Open **Power BI Desktop**
2. Click **Get Data** → **Azure** → **Azure Databricks**
3. Enter the following details obtained from Databricks:

- **Server Hostname**
- **HTTP Path**
(Found under Databricks → SQL Warehouses → Serverless → Connection Details)

4. Select **Microsoft Account** as the authentication method

5. Click Sign In to proceed

This establishes a secure live connection between Power BI and Databricks without exporting data manually.

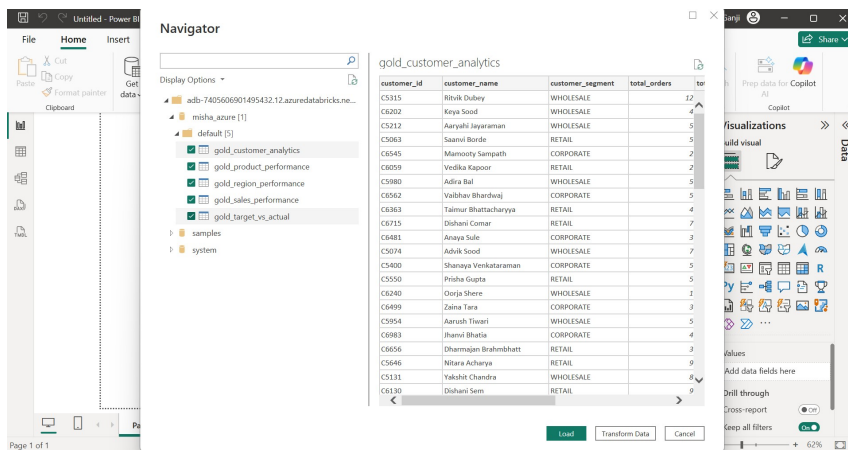


4.3 Authenticate Power BI with Microsoft Azure Account

When prompted by Power BI:

1. Sign in using your **Microsoft Azure account**
2. Grant the required permissions to access Databricks data
3. Establish a secure live connection between Databricks and Power BI

Once authenticated, the Gold tables become available for report creation.



PowerBI

- Power BI is used as the visualization and reporting layer of the project to convert Gold-layer analytics data into interactive dashboards.
- Connects directly to Gold tables for reporting
- Displays key business KPIs and trends
- Supports filters, slicers, and drill-down analysis
- Enables business users to explore data interactively
- Power BI helps transform processed data into actionable insights for decision-makers.

4.4 Create Power BI Dashboards

Using the connected Gold layer datasets, three interactive dashboards are created:

- **Sales Performance Dashboard**
Overall revenue, profit, sales trends, and KPI metrics



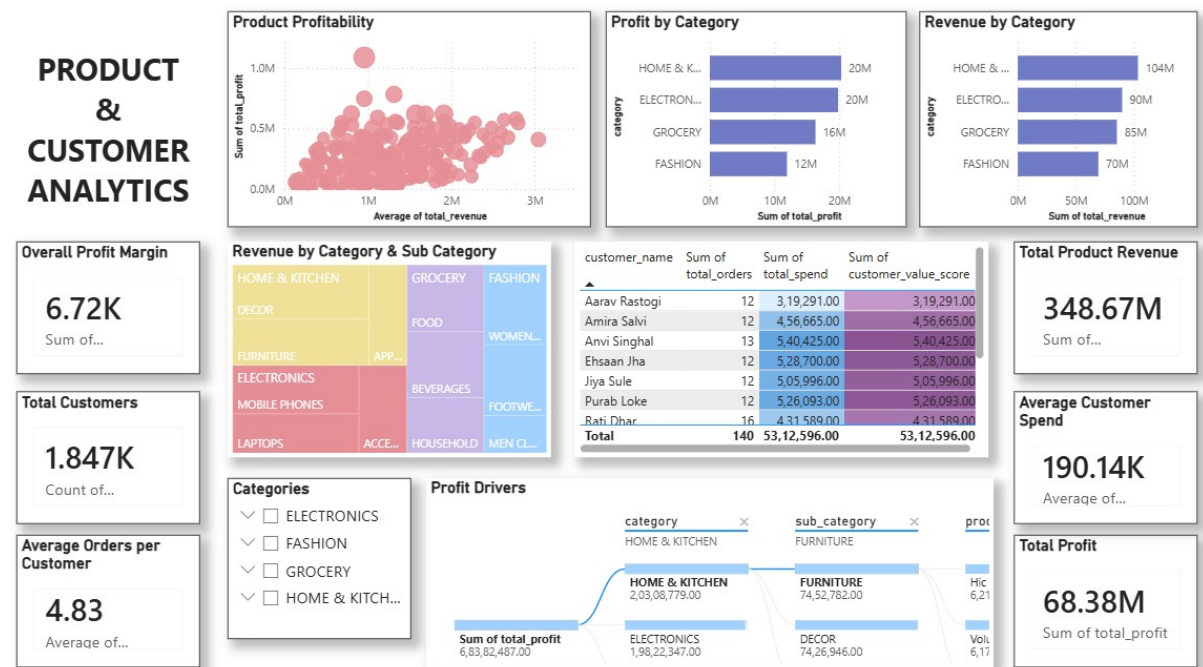
- Regional & Store Performance Dashboard**

Region-wise and store-level comparisons with geographic insights



- Product & Customer Analytics Dashboard**

Product profitability, customer segmentation, and purchase behavior analysis



These dashboards provide actionable insights for business stakeholders and support data-driven decision-making.

Testing & Data Validation

The pipeline includes validation checks at each stage to ensure data quality:

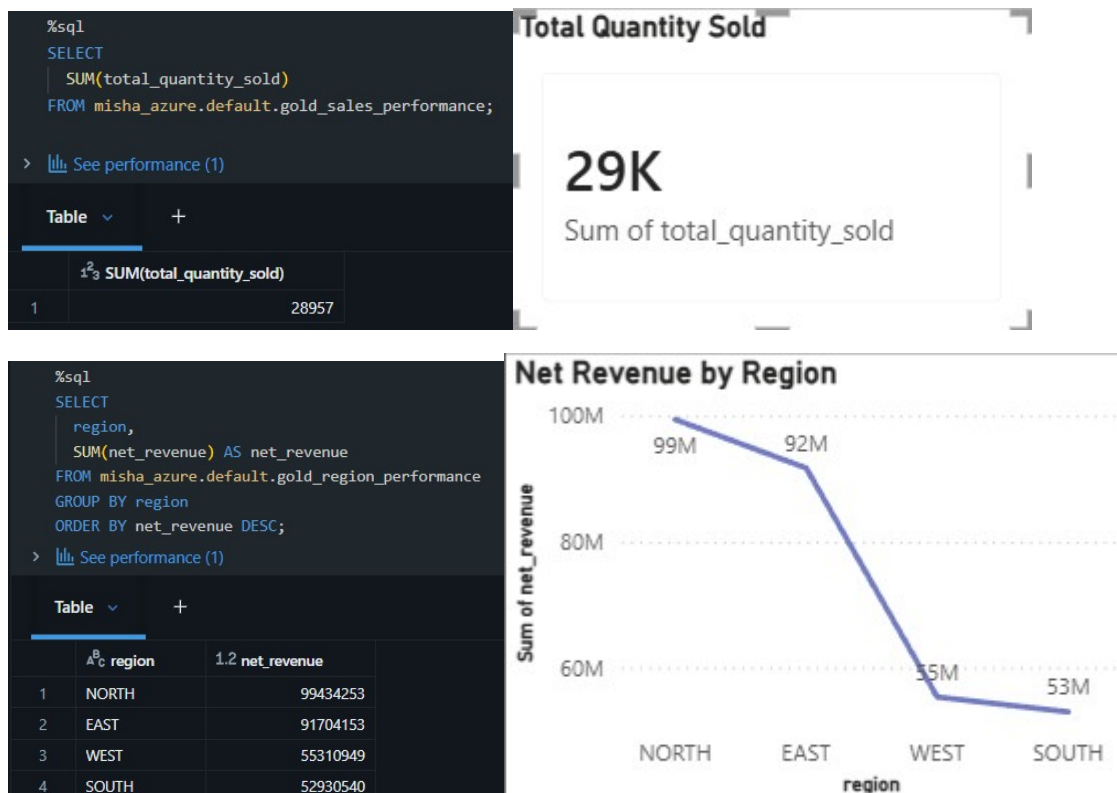
- Record count reconciliation between Bronze, Silver, and Gold layers
- Null, duplicate, and invalid value checks in the Silver layer
- Referential integrity validation across sales, products, customers, and stores
- Verification of KPI calculations in the Gold layer
- Cross-validation of Power BI dashboard metrics against Gold tables

KPI Validation

Key business KPIs were defined using validated Silver-layer data and materialized in the Gold layer. Metrics such as revenue, profit, profit margin, return rate, and target achievement percentage were calculated using consistent and business-approved formulas.

KPI validation was performed verifying consistency between Gold tables and Power BI dashboards. These steps ensured accuracy and reliability of all reported metrics.

Examples



Results & Conclusion

- Clear business insights were identified from the retail sales data, including:
 - Sales trends across regions and time periods
 - High-performing and underperforming products and categories

- Variations in store-level and regional performance
 - Customer purchase patterns and segmentation
- Profitability analysis highlighted:
 - Regions and products contributing the highest profit
 - Impact of returns and discounts on net revenue
 - Differences between actual sales and defined sales targets

Key Outcomes

- Successfully transformed raw, inconsistent retail datasets into clean, analytics-ready data layers
- Implemented a scalable Medallion Architecture (Bronze, Silver, Gold)
- Created reliable business KPIs such as revenue, profit, and target achievement
- Automated ETL workflows using Apache Airflow with proper task dependencies and logging
- Delivered interactive and insightful Power BI dashboards for business stakeholders